ictPRO

Vzdělávací materiály ICT Pro

alter for the better

ICT kurzy
školení & workshopy

Soft skills
školení & semináře

Vibe Coding a tvorby aplikací bez programování

Petr Rozkošný

Obsah

1 Srovnání Claude produktů — vrstvy kontextu ... 3
2 Claude Code — instalace a základní workflow ... 6
3 Prompting best practices: Obecné principy ... 10
4 Prompting best practices: Extended thinking ... 12
5 CLAUDE.md — anatomie, hierarchie a best practices ... 14
6 Permissions a bezpečnost v Claude Code ... 17
7 Redukce halucinací a zvýšení konzistence výstupu ... 20
8 Skills v Claude Code — co to je, jak je vytvářet a sdílet ... 25
9 Sub-agenti v Claude Code — co to jsou, jak je vytvářet a používat ... 30
10 Artifacts v Claude — co to je, jak je vytvářet a sdílet ... 35

1 Srovnání Claude produktů — vrstvy kontextu

1.1 Tři produkty

Vrstva	Claude.ai / Desktop App (Chat)	Claude.ai — Projects	Claude Code (CLI)
Paměť (persistentní)	Memory summary — automaticky generovaná z historie chatů, uživatel může editovat. Globální pro všechny chaty mimo projekty.	Vlastní **izolovaná paměť per projekt** (oddělená od globální).	`CLAUDE.md` soubory v hierarchii (globální `~/.claude/CLAUDE.md` ⟶ projektový `./CLAUDE.md` ⟶ vnořené) + Auto Memory (`MEMORY.md`)
Systémové instrukce	System prompt od Anthropic (uživatel ho nevidí/neovlivní).	System prompt + **custom instructions** nastavené uživatelem per projekt.	System prompt + `CLAUDE.md` instrukce (plně pod kontrolou uživatele).
Knowledge base / Dokumenty	Pouze přílohy v rámci jednoho chatu (nemají persistenci).	Soubory nahrané do projektu — Claude je vidí ve **všech chatech** v rámci projektu (až 200k tokenů, s RAG až 10×).	Přímý přístup k souborovému systému (čte soubory z repozitáře on-demand).
Styl / Preference	User Preferences + Style (globální nastavení).	User Preferences + Style (globální) + custom instructions projektu.	Styl se řeší přes `CLAUDE.md` instrukce.
Nástroje (tools)	Web search, artifacts, code execution, image gen (vestavěné).	Stejné jako chat.	Bash, Read, Write, Edit, Search, Glob, Grep + **MCP servery** (GitHub, DB, Slack…) + slash commands.
Pluginy / MCP	Integrace (Google Drive, Gmail…) — Anthropic connectors.	Stejné connectors jako chat.	MCP servery — uživatel konfiguruje vlastní (stdio/SSE/HTTP). Plně rozšiřitelné.
Iterace agenta	Jednorázová odpověď (ne agentic loop).	Jednorázová odpověď.	**Plný agentic loop** — tool_use ⟶ tool_result ⟶ reasoning ⟶ opakuje dokud nesplní úkol.
Konverzační historie	Chaty jsou oddělené, ale memory + search past chats umožňuje referencovat.	Chaty v rámci projektu sdílí knowledge base a instrukce, ale každý chat je jinak oddělený.	Session-based. `/compact` shrne kontext. Mezi sessions přes `CLAUDE.md` a Auto Memory.

1.2 Klíčové rozdělení

1. **Claude.ai chat** = nejjednodušší vrstva, minimum persistence
2. **Claude.ai Projects** = přidává izolovaný workspace s vlastní pamětí, knowledge base a custom instrukcemi
3. **Claude Code** = plný agentic režim s přístupem k filesystému, MCP rozšiřitelností a agentic loop
4. **Desktop App** ≈ Claude.ai (stejné funkce, jiný klient — web vs. nativní aplikace)

1.3 Connectors (claude.ai)

Connectors jsou **předkonfigurované MCP servery**, které propojují Claude s externími službami přímo v claude.ai, Claude Desktop a Claude Mobile. Uživatel je zapíná v Settings ⟶ Connectors bez technické konfigurace.

1.3.1 Dostupné connectors

Connector	Co umožňuje
Gmail	Hledání a čtení emailů, porozumění kontextu korespondence
Google Calendar	Zobrazení událostí, porozumění rozvrhu
Google Drive	Hledání a čtení dokumentů, spreadsheetů, prezentací
Slack	Hledání v kanálech, DMs, sdílených souborech
GitHub	Hledání v repozitářích, issues, PRs
Microsoft 365	SharePoint, OneDrive, Outlook, Teams
Notion, Asana, Linear, Jira…	Partnerské connectors — práce s úkoly, dokumenty, projekty

Plus jakékoliv **custom connectors** postavené na remote MCP standardu — vlastní nebo třetích stran.

1.3.2 Jak zapnout

Individuální uživatel (Pro/Max):

1. Settings ⟶ Connectors
2. Klik "Connect" u požadované integrace
3. Autentizace přes OAuth v prohlížeči

Team/Enterprise organizace:

1. Owner: Organization settings ⟶ Connectors ⟶ "Enable"
2. Jednotliví členové: Settings ⟶ Connectors ⟶ autentizace

1.3.3 Jak používat

V chatu nebo v projektu jednoduše odkazujte na připojené služby:

- `"Najdi v Google Drive dokument o Q3 rozpočtu"`
- `"Zkontroluj můj kalendář na zítra a navrhni, kdy mám volno na hodinovou schůzku"`
- `"Projdi emaily od Petra za poslední týden a shrň hlavní témata"`
- `"Podívej se na otevřené issues v repozitáři backend-api"`

Claude automaticky použije příslušný connector.

1.3.4 Connectors vs. MCP v Claude Code

	Connectors (claude.ai)	MCP servery (Claude Code)
Konfigurace	Pár kliků v UI	.mcp.json, technická konfigurace
Cílová skupina	Běžní uživatelé	Vývojáři
Rozšiřitelnost	Předem dané + custom MCP URL	Neomezená (stdio, SSE, HTTP)
Kde funguje	claude.ai, Desktop app, mobilní app	Terminál, VS Code, JetBrains, Desktop
Autentizace	OAuth přes prohlížeč	Tokeny, env variables, lokální konfigurace

Connectors jsou v podstatě MCP servery hostované poskytovateli služeb (Google, Slack, Asana…), ke kterým se claude.ai připojuje přes remote MCP protokol. Claude Code používá stejný MCP standard, ale s plnou kontrolou nad konfigurací.

1.3.5 Enterprise Search

Team a Enterprise plány mají přístup k **Enterprise Search** — předkonfigurovaný projekt, který prohledává všechny připojené connectors najednou (Slack + Google Drive + M365 + další). Přístup přes “Ask Your Org” v levém menu.

2 Claude Code — instalace a základní workflow

2.1 Co je Claude Code

Claude Code je **agentic coding assistant** v terminálu. Na rozdíl od chatbota, který odpovídá a čeká, Claude Code:

- čte vaše soubory
- spouští příkazy
- edituje kód
- autonomně řeší problémy

Popíšete, co chcete — Claude zjistí, jak to udělat.

2.2 Kde Claude Code běží

Prostředí	Popis
Terminál (CLI)	Hlavní rozhraní. Plná funkcionalita.
VS Code	Extension s integrovaným diff view
JetBrains	Plugin pro IntelliJ, PyCharm, WebStorm
Desktop app	GUI s vizuálním diff review, session management
Web	claude.ai/code — bez lokální instalace, práce v cloudu
Slack	@Claude v kanálech, automatické spouštění Code sessions

Všechna prostředí sdílejí stejný engine — CLAUDE.md, settings, MCP servery fungují napříč všemi.

2.3 Instalace

2.3.1 npm (doporučeno)

```
npm install -g @anthropic-ai/claude-code
```

2.3.2 Homebrew (macOS)

```
brew install claude-code
```

Homebrew verze se **neaktualizuje automaticky** — je třeba `brew upgrade claude-code`.

2.3.3 Windows

Vyžaduje Git for Windows. Nainstalujte ho předem.

2.3.4 Ověření

```
claude --version
```

2.3.5 Přihlášení

Při prvním spuštění se otevře prohlížeč pro OAuth přihlášení. Potřebujete:

- Claude subscription (Pro, Max, Teams, Enterprise), nebo
- Anthropic Console účet, nebo
- přístup přes podporovaného cloud providera (AWS Bedrock, Vertex AI)

2.4 První spuštění

```
cd /cesta/k/projektu
claude
```

Claude Code se spustí v aktuálním adresáři. Vidí:

- **Soubory** ve vašem adresáři a podadresářích
- **Terminál** — cokoli, co můžete spustit z příkazové řádky
- **Git stav** — aktuální branch, uncommitted changes, historie commitů
- **CLAUDE.md** — persistentní instrukce (pokud existují)

2.5 Základní interakce

Mluvte s Claude jako s kolegou. Popište, co chcete dosáhnout:

Místo: `"fix the bug"`
Zkuste: `"oprav login bug, kdy uživatel vidí prázdnou stránku po zadání špatného hesla"`

Komplexní zadání:

```
1. vytvoř novou databázovou tabulku pro user profiles
2. vytvoř API endpoint pro get a update
3. postav webovou stránku, kde si uživatel upraví profil
```

2.6 Schvalování změn

Claude Code **vždy žádá o povolení** před úpravou souborů. Máte na výběr:

Akce	Co dělá
y (Yes)	Schválí konkrétní změnu
n (No)	Odmítne, Claude navrhne alternativu
Accept all	Schválí vše pro zbytek session (přepnutí módu)

2.6.1 Permission modes

Mód	Popis	Kdy použít
Ask permissions	Claude se ptá na každou akci	Default, maximální kontrola
Accept edits	Automaticky schvaluje editace souborů	Když důvěřujete Claude, ale chcete vidět bash
Plan mode	Claude plánuje, ale neprovádí	Code review, analýza, plánování
Bypass permissions	Bez schvalování	Pouze v izolovaném prostředí (kontejner, VM)

2.7 Klíčové slash příkazy

Příkaz	Co dělá
`/help`	Nápověda
`/model`	Změna modelu (Opus, Sonnet, Haiku)
`/compact`	Manuální compaction (shrnutí kontextu)
`/compact <focus>`	Compaction se zaměřením ("focus on API changes")
`/clear`	Smaže historii, čistý start
`/context`	Zobrazí využití context window
`/memory`	Editor CLAUDE.md a auto memory
`/mcp`	Správa MCP serverů, zobrazení token cost
`/permissions`	Správa povolení pro nástroje
`/init`	Inicializuje CLAUDE.md pro aktuální projekt
`/resume`	Obnoví předchozí session
`/status`	Zobrazí aktivní konfiguraci a zdroje

2.8 Nástroje, které Claude Code používá

Nástroj	Co dělá
Read	Čte soubory
Write	Zapisuje/vytváří soubory
Edit	Edituje existující soubory (targeted changes)
Bash	Spouští shell příkazy (build, test, git…)
Grep	Hledá v souborech
Glob	Najde soubory podle vzoru
WebFetch	Stáhne obsah z URL
Task	Deleguje úkol na subagenta

2.9 Session a kontext

Každá session je **nezávislá** — nová session začíná s čistým context window, bez konverzační historie z předchozích sessions.

Co **přežívá** mezi sessions:
- CLAUDE.md (čte se z disku)
- Auto memory (MEMORY.md)
- Soubory na disku (samozřejmě)
- Git historie

Co **nepřežívá**:
- Konverzační historie
- Instrukce zadané v chatu
- Tool results z předchozí session

Proto je CLAUDE.md tak důležitý — je to jediný spolehlivý způsob, jak předávat kontext mezi sessions.

2.10 Typický workflow pro začátečníky

2.10.1 1. Explorace

```
> vysvětli mi strukturu tohoto projektu
> co dělá soubor src/auth/middleware.ts?
> jaké testy máme pro autentizaci?
```

2.10.2 2. Plánování

```
> navrhni plán, jak přidat funkci pro reset hesla
```

2.10.3 3. Implementace

```
> implementuj plán z předchozího kroku
```

2.10.4 4. Testování

```
> spusť testy a oprav to, co selhalo
```

2.10.5 5. Commit

```
> vytvoř commit s popisnou zprávou
```

2.11 Tipy pro začátečníky

2.11.1 Přerušujte a korigujte

Můžete Claude kdykoliv přerušit (Esc). Pokud jde špatným směrem, zastavte ho a upřesněte zadání.

2.11.2 Začněte s /init

V novém projektu spusťte `/init` — Claude vytvoří počáteční CLAUDE.md s build příkazy a konvencemi.

2.11.3 Používejte /context

Sledujte, kolik kontextu zbývá. Pod 30 % začíná degradace.

2.11.4 Čistý start mezi úkoly

Po dokončení nesouvisejícího úkolu použijte `/clear` — začnete s čistým kontextem.

2.11.5 Esc + Esc

Dvojitý Esc zastaví Claude a umožní přidat instrukce nebo opravit směr.

3 Prompting best practices: Obecné principy

3.1 1. Buďte explicitní

Claude 4.x modely reagují nejlépe na jasné, konkrétní instrukce. Pokud chcete určité chování, řekněte to přímo — Claude nebude "hádat" vaše záměry tak jako starší modely.

Místo:

> **POZNÁMKA**
>
> „Vytvoř analytický dashboard"

Raději:

> **TIP**
>
> „Vytvoř analytický dashboard. Zahrň co nejvíc relevantních funkcí a interakcí. Jdi nad rámec základů a vytvoř plně vybavenou implementaci."

3.2 2. Přidejte kontext (vysvětlete proč)

Vysvětlení motivace za instrukcí pomůže Claude lépe pochopit vaše cíle. Neříkejte jen *co*, ale i *proč*.

Místo:

> **POZNÁMKA**
>
> „Odpovídej stručně"

Raději:

> **TIP**
>
> „Odpovídej stručně — výstupy budou součástí technické dokumentace, kde čtenáři potřebují rychle najít konkrétní informaci."

Claude je dostatečně inteligentní, aby z vysvětlení generalizoval na další situace.

3.3 3. Pozor na příklady a detaily

Claude 4.x modely věnují příkladům velkou pozornost. Ujistěte se, že vaše příklady:

- Odpovídají chování, které chcete **podpořit**
- Minimalizují chování, které chcete **potlačit**

Špatný příklad v promptu může stačit k tomu, aby Claude opakoval nechtěný vzor.

3.4 4. Řízení formátu odpovědí

Čtyři techniky, které fungují nejlépe:

3.4.1 a) Říkejte co dělat, ne co nedělat

Místo: „Nepoužívej markdown“
Raději: „Odpověz v plynulé próze bez formátování.”

3.4.2 b) Použijte XML tagy jako formátový signál

```
Piš prózové části odpovědi v <smoothly_flowing_prose_paragraphs> tazích.
```

3.4.3 c) Styl promptu ovlivňuje styl odpovědi

Pokud váš prompt sám obsahuje markdown, Claude bude inklinovat k markdown odpovědi. Chcete-li prózu, formulujte prompt jako prózu.

3.4.4 d) Explicitní instrukce pro formátování

Pro minimalizaci markdown a bullet pointů:

```
<avoid_excessive_markdown_and_bullet_points>
Při psaní reportů, dokumentace a analýz piš v jasné, plynulé próze
s odstavci a celými větami. Používej markdown pouze pro `inline code`,
code blocky a jednoduché nadpisy.

NEPOUŽÍVEJ číslované nebo odrážkové seznamy, pokud:
a) se nejedná o skutečně diskrétní položky, kde je seznam nejlepší volba
b) uživatel explicitně nepožádal o seznam

Cílem je čitelný, plynulý text, který čtenáře přirozeně vede myšlenkami.
</avoid_excessive_markdown_and_bullet_points>
```

3.5 5. Komunikační styl Claude 4.x

Claude 4.5 modely mají přirozenější, stručnější styl:

- **Přímější** — reportuje fakta místo sebeoslavných updatů
- **Konverzačnější** — méně strojový, více koloquiální
- **Méně ukecaný** — může přeskočit detailní shrnutí po tool calls

Pokud chcete, aby Claude reportoval průběh práce:

```
After completing a task that involves tool use, provide a quick summary
of the work you've done.
```

4 Prompting best practices: Extended thinking

4.1 Co je extended thinking

Extended thinking umožňuje Claude „přemýšlet nahlas“ v interním thinking bloku předtím, než odpoví. Thinking blok je viditelný (na API nebo v Claude Code), ale není součástí odpovědi uživateli.

Hodí se pro:
- Komplexní vícekrokové uvažování
- Reflexi po tool use (hodnocení kvality výsledků)
- Matematické a logické problémy
- Analýzu kódu před návrhem řešení
- Plánování před implementací

4.2 Interleaved thinking

Claude 4.x modely podporují **interleaved thinking** — myšlení nejen na začátku, ale i **mezi kroky tool use**. To znamená, že Claude může:

1. Přečíst soubor (tool call)
2. **Zamyslet se** nad tím, co přečetl (thinking)
3. Spustit test (tool call)
4. **Reflektovat** výsledky testu (thinking)
5. Navrhnout opravu na základě reflexe

4.2.1 Prompt pro interleaved thinking

```
After receiving tool results, carefully reflect on their quality
and determine optimal next steps before proceeding. Use your thinking
to plan and iterate based on this new information, and then take
the best next action.
```

4.3 Citlivost na slovo „think“

Když je extended thinking **vypnuté**, Claude Opus 4.5 je extrémně citlivý na slovo „think“ a jeho varianty — může to vést k neočekávanému chování.

Doporučené nahrazení:

Nepoužívejte	Použijte
think about	consider, evaluate
think through	work through, analyze
think step by step	reason step by step, work through carefully
I think	I believe, I expect
let me think	let me consider, let me evaluate

Toto omezení **neplatí** když je extended thinking zapnuté.

4.4 Kdy zapnout / vypnout

Situace	Extended thinking
Komplexní architekturní rozhodnutí	Zapnout
Debugging složitých bugů	Zapnout
Jednoduché file edity	Zbytečné (zvyšuje latenci a cost)
Rutinní operace (git, file read)	Zbytečné
Plánování multi-step úkolů	Zapnout
Code review	Zapnout

4.5 V Claude Code

V Claude Code se extended thinking ovládá přes:

- `/model` — výběr modelu (thinking modely mají suffix)
- Nastavení v `/config`
- Automaticky se aktivuje u komplexních úkolů (závisí na konfiguraci)

5 CLAUDE.md — anatomie, hierarchie a best practices

5.1 Co je CLAUDE.md

`CLAUDE.md` je markdown soubor s instrukcemi, pravidly a kontextem, který Claude Code čte na začátku každé session. Je to **nejdůležitější konfigurační mechanismus** — persistentní instrukce, které přežijí compaction i nové session.

Na rozdíl od konverzačních instrukcí (které se ztrácí při compaction), CLAUDE.md se čte z disku vždy znovu.

5.2 Hierarchie paměťových souborů

Claude Code hledá CLAUDE.md soubory ve stromové struktuře. Specifičtější instrukce mají přednost před obecnějšími.

5.2.1 1. Uživatelský CLAUDE.md (globální)

`~/.claude/CLAUDE.md`

Platí pro **všechny projekty**. Sem patří osobní preference: styl kódu, preferovaný jazyk, obecné workflow.

5.2.2 2. Projektový CLAUDE.md (root repozitáře)

`<projekt>/CLAUDE.md`

Commituje se do gitu → sdílený celým týmem. Obsahuje pravidla projektu: architektura, konvence, preferované knihovny, build příkazy.

5.2.3 3. CLAUDE.local.md (osobní, ignorovaný gitem)

`<projekt>/CLAUDE.local.md`

Automaticky přidán do `.gitignore`. Pro osobní per-project preference, které nechcete sdílet (cesty, lokální nastavení, osobní workflow).

5.2.4 4. Podadresářové CLAUDE.md

`<projekt>/src/api/CLAUDE.md`

Načítá se **on-demand** — pouze když Claude čte soubory v daném adresáři. Pro instrukce specifické pro modul (API konvence, testovací vzory).

5.2.5 5. Modulární pravidla (.claude/rules/)

```
<projekt>/.claude/rules/
├── testing.md          # Pravidla pro testy
├── api-*.md            # Glob pattern pro API soubory
└── src/frontend/*.md   # Path-specific pravidla
```

Pravidla podporují glob patterns a načítají se, když Claude pracuje s odpovídajícími soubory.

5.2.6 Priorita při konfliktech

Managed (enterprise) > User (~/) > Project (root) > Podadresáře

Claude při konfliktech používá úsudek, přičemž specifičtější instrukce typicky mají přednost.

5.3 Auto Memory

Vedle CLAUDE.md, které **vy píšete pro Claude**, existuje auto memory — poznámky, které **Claude píše sám pro sebe** na základě toho, co objeví během práce.

5.3.1 Jak funguje

- Zapnutá ve výchozím stavu (toggle přes `/memory`)
- Každý projekt má vlastní adresář: `~/.claude/projects/<projekt>/memory/`
- Cesta `<projekt>` se odvozuje z git root → všechny podadresáře stejného repa sdílejí jednu auto memory

5.3.2 Struktura

```
~/.claude/projects/<projekt>/memory/
├── MEMORY.md           # Hlavní soubor – prvních 200 řádků se načte vždy
├── debugging.md        # Detailní poznámky o debugování
├── api-conventions.md  # API rozhodnutí
└── ...                 # Tematické soubory
```

5.3.3 Co se načítá

- **MEMORY.md** — prvních 200 řádků se vloží do system promptu na začátku session
- **Tematické soubory** (debugging.md, patterns.md…) — Claude je čte on-demand, když potřebuje
- Obsah nad 200 řádků MEMORY.md se **nenačítá** automaticky

5.3.4 Ruční ukládání

Řekněte Claude přímo: `"zapamatuj si, že používáme pnpm, ne npm"` nebo `"ulož do memory, že API testy vyžadují lokální Redis"`.

5.3.5 Vypnutí

```
export CLAUDE_CODE_DISABLE_AUTO_MEMORY=1   # Vynutit vypnutí
export CLAUDE_CODE_DISABLE_AUTO_MEMORY=0   # Vynutit zapnutí
```

5.4 CLAUDE.md importy

CLAUDE.md soubory mohou importovat další soubory syntaxí `@cesta`:

```
See @README for project overview and @package.json for available npm commands.

# Additional Instructions
@docs/git-instructions.md
```

Relativní i absolutní cesty jsou povoleny. Relativní cesty se rozlišují vůči souboru obsahujícímu import, ne vůči working directory.

5.5 Co psát do CLAUDE.md

5.5.1 Patří tam

- **Build a test příkazy**: `npm run test`, `make build`, `pytest -x`
- **Architekturní rozhodnutí**: "Používáme hexagonal architecture, domain logic v src/domain/"
- **Konvence kódu**: "TypeScript strict mode, žádné any, max 30 řádků na funkci"
- **Preferované knihovny**: "Pro HTTP používáme axios, ne fetch"
- **Git workflow**: "Feature branches, squash merge, conventional commits"
- **Compact instructions**: Sekce "Compact Instructions" říká, co zachovat při compaction
- **Kritická pravidla**: Věci, které se nesmí ztratit ani po 10 compaction cyklech

5.5.2 Nepatří tam

- Detailní API dokumentace (přesunout do skills nebo podpůrných souborů)
- Obsah přes 500 řádků (Claude to zpracovává v každém requestu → zbytečně plní context)
- Citlivé údaje (hesla, API klíče, tokeny)
- Informace, které se rychle mění (používejte auto memory)

5.6 Praktický příklad

```
# CLAUDE.md — Projekt: E-shop API

## Build & Test
- `npm run dev` — vývojový server (port 3000)
- `npm run test` — spustí vitest
- `npm run lint` — eslint + prettier check

## Architektura
- Express.js + TypeScript (strict mode)
- PostgreSQL přes Prisma ORM
- Autentizace: JWT v httpOnly cookies
- Struktura: src/routes/ → src/services/ → src/repositories/

## Konvence
- Funkce: camelCase, max 30 řádků
- Soubory: kebab-case.ts
- Commit messages: conventional commits (feat:, fix:, chore:)
- Error handling: custom AppError class, nikdy holé throw

## Git workflow
- Feature branches z `develop`
- PR review povinné, squash merge
- `main` = production, `develop` = staging

## Compact Instructions
Při compaction zachovej: build příkazy, architekturu, konvence,
git workflow. Tyto informace jsou kritické pro správné fungování.
```

5.7 Správa přes /memory

Příkaz `/memory` otevře interaktivní selektor, kde můžete:

- Editovat CLAUDE.md soubory (uživatelské i projektové)
- Editovat auto memory (MEMORY.md)
- Zapnout/vypnout auto memory

5.8 Best practices

5.8.1 Držte CLAUDE.md pod 500 řádků

Vše nad toto přesuňte do skills (on-demand) nebo podpůrných souborů s @importem.

5.8.2 Používejte sekci "Compact Instructions"

Claude ji respektuje při compaction — říká, co zachovat.

5.8.3 Commitujte projektový CLAUDE.md

Celý tým pak sdílí stejné konvence. Osobní preference do CLAUDE.local.md.

5.8.4 Auto memory nechte zapnutou

Claude se učí z chyb a ukládá si poznatky (build příkazy, workaroundy, vzory). Nemusíte to dělat ručně.

5.8.5 Kontrolujte /context

Příkaz `/context` ukáže, kolik místa zabírají CLAUDE.md, skills, MCP servery a konverzace. Pokud je CLAUDE.md příliš velký, optimalizujte.

6 Permissions a bezpečnost v Claude Code

6.1 Proč na permissions záleží

Claude Code má přístup k vašemu souborovému systému a terminálu — může číst, psát, mazat soubory a spouštět libovolné příkazy. Permissions systém je vrstva, která toto kontroluje.

Princip je jednoduchý: **deny → allow → ask**. Pravidla se vyhodnocují v tomto pořadí. První shoda vyhrává, deny pravidla mají vždy přednost.

6.2 Permission modes

Mód	Chování	Kdy použít
default (Ask)	Claude se ptá na neznámé operace	Každodenní práce, maximální kontrola
acceptEdits	Automaticky schvaluje editace souborů	Důvěřujete editacím, ale chcete kontrolovat bash
plan	Claude plánuje, ale neprovádí	Code review, analýza, plánování
bypassPermissions	Žádné kontroly	**Pouze** v izolovaném prostředí (kontejner, VM)

Mód nastavíte v settings.json:

```
{
  "permissions": {
    "defaultMode": "default"
  }
}
```

Nebo přepnete za běhu v Desktop app.

6.3 Pravidla oprávnění

6.3.1 Syntax

Pravidla mají formát `Nástroj` nebo `Nástroj(specifikátor)`:

```
Bash                    → všechna bash volání
Bash(npm run test *)    → příkazy začínající "npm run test "
Read(./.env)            → čtení konkrétního souboru
Read(./secrets/**)      → čtení čehokoliv v secrets/ (rekurzivní)
Edit                    → všechny editace
WebFetch(domain:github.com) → fetch pouze z github.com
```

6.3.2 Praktický příklad settings.json

```
{
  "permissions": {
    "allow": [
      "Bash(npm run lint)",
      "Bash(npm run test *)",
      "Read(~/.zshrc)"
    ],
    "deny": [
      "Bash(curl *)",
      "Read(./.env)",
      "Read(./.env.*)",
      "Read(./secrets/**)"
    ]
  }
}
```

Tato konfigurace:

- automaticky povolí lint a testy
- zablokuje curl (síťový přístup přes bash)
- zablokuje čtení .env souborů a secrets adresáře
- na vše ostatní se zeptá (default behavior)

6.3.3 Správa přes /permissions

Interaktivní příkaz `/permissions` (nebo `/allowed-tools`) umožňuje přidat pravidla za běhu. Claude si je pamatuje pro aktuální projekt.

6.4 Hierarchie settings

Settings se skládají z více vrstev (vyšší priorita přepisuje nižší):

```
1. Managed settings (enterprise)     ← nejvyšší priorita, nejde přepsat
2. Command line argumenty
3. Local project (.claude/settings.local.json)
4. Shared project (.claude/settings.json)
5. User (~/.claude/settings.json)    ← nejnižší priorita
```

Pokud user settings povolují `Bash(npm run *)`, ale project settings to zakazují, project settings vyhrávají.

Array settings (jako allow/deny) se **mergují** přes vrstvy — nižší priorita může přidat položky, aniž by přepsala vyšší.

6.5 Co Claude Code vidí vs. nevidí

6.5.1 Vidí (a může přistupovat)

- Soubory v aktuálním adresáři a podadresářích
- Soubory jinde na disku (s vaším povolením)
- Výstup příkazů v terminálu
- Git stav, branch, diff
- MCP servery (pokud nakonfigurované)

6.5.2 Nevidí (bez explicitního povolení)

- Soubory zablokované deny pravidly
- Síťové zdroje (pokud nemá povolený curl/wget nebo WebFetch)
- Procesy běžící mimo session

6.6 Sandboxing

Sandbox poskytuje **OS-level** izolaci, která omezuje bash příkazy na úrovni operačního systému. Kombinace permissions + sandbox dává defense-in-depth:

- **Permission deny rules** — zabrání Claude vůbec pokusit se o přístup
- **Sandbox** — zabrání bash příkazům dosáhnout na zdroje mimo definované hranice, i kdyby prompt injection obešel Claudeovo rozhodování

Sandbox se konfiguruje v settings.json a omezuje filesystem a síťový přístup bash příkazů.

6.7 Bezpečnostní best practices

6.7.1 1. Nikdy neukládejte secrets do CLAUDE.md

Používejte environment variables mimo Claude Code session.

6.7.2 2. Deny pravidla pro citlivé soubory

```
{
  "permissions": {
    "deny": [
      "Read(./.env*)",
      "Read(./secrets/**)",
      "Read(**/*.pem)",
      "Read(**/*.key)"
    ]
  }
}
```

6.7.3 3. Omezte síťový přístup

```
{
  "permissions": {
    "deny": ["Bash(curl *)", "Bash(wget *)"],
    "allow": ["WebFetch(domain:github.com)"]
  }
}
```

6.7.4 4. Bypass permissions pouze v izolaci

`bypassPermissions` mode používejte **výhradně** v kontejnerech nebo VM. Nikdy na lokálním stroji s přístupem k produkčním datům.

6.7.5 5. Pro týmy: sdílená pravidla přes git

Commitněte `.claude/settings.json` do repozitáře — celý tým bude mít stejná pravidla. Osobní úpravy do `.claude/settings.local.json` (automaticky v .gitignore).

6.7.6 6. Enterprise: managed settings

Administrátoři mohou deploynout settings, které uživatelé ani projekty nemůžou přepsat. Vhodné pro organizační bezpečnostní politiky.

6.8 Diagnostika

```
claude /status    # Zobrazí aktivní settings a jejich zdroj
claude --debug    # Debug mód – zobrazí chyby při načítání
claude /doctor    # Diagnostika instalace a funkcionalit
```

`/status` ukáže každou konfigurační vrstvu (managed, user, project) a její původ.

7 Redukce halucinací a zvýšení konzistence výstupu

7.1 Část 1: Redukce halucinací

Halucinace = model generuje text, který je fakticky nesprávný nebo nekonzistentní s poskytnutým kontextem. I nejpokročilejší modely (včetně Claude Opus 4.6) mohou halucinovat. Důležité je vědět, kdy k tomu dochází a jak to minimalizovat.

7.1.1 Kdy Claude halucinuje nejčastěji

- **Mezery ve znalostech** — model „vyplní“ informaci, kterou nemá
- **Nejednoznačný vstup** — prompt umožňuje více interpretací
- **Dlouhé dokumenty** — u 20K+ tokenů klesá přesnost extrakce faktů
- **Obecné znalosti vs. kontext** — model může přidat informace z tréninku, které neodpovídají dodaným datům
- **Po compaction** v Claude Code — ztráta detailů z dřívějších zpráv

7.1.2 Základní techniky

7.1.2.1 1. Dovolte Claude říct „nevím“

Explicitně dejte Claude povolení přiznat nejistotu. Tato jednoduchá technika drasticky sníží falešné informace.

```
Analyzuj tento dokument a odpověz na mé otázky.
Pokud si odpovědí nejsi jistý nebo informace v dokumentu není,
řekni "Na základě poskytnutého dokumentu tuto informaci nemohu potvrdit."
Nevymýšlej si.
```

Bez tohoto povolení Claude inklinuje k tomu generovat „plausibilní“ odpověď místo přiznání, že neví.

7.1.2.2 2. Přímé citace pro faktické ukotvení

U dlouhých dokumentů (>20K tokenů) nechte Claude nejprve extrahovat doslovné citáty a teprve pak provést úlohu. To ukotvuje odpovědi v reálném textu.

```
Nejprve extrahuj doslovné citáty z dokumentu, které jsou relevantní
k otázce. Potom na základě POUZE těchto citátů formuluj odpověď.
Pokud žádný relevantní citát nenajdeš, řekni to.
```

Doslovné citáty můžete ověřovat string matchingem nebo výpočtem překryvu s originálním dokumentem.

7.1.2.3 3. Verifikace pomocí citací

Claude musí každé tvrzení podložit citací. Pokud citaci nenajde, musí tvrzení stáhnout.

```
Pro každé tvrzení ve své odpovědi:
1. Uveď přímou citaci z dokumentu, která ho podporuje
2. Pokud citaci nenajdeš, tvrzení NEUVÁDĚJ
3. Označ, odkud citace pochází (sekce, stránka)
```

7.1.3 Pokročilé techniky

7.1.3.1 4. Chain-of-thought verifikace

Nechte Claude vysvětlit svůj postup krok za krokem předtím, než dá finální odpověď. Odhalí to chybnou logiku nebo předpoklady.

```
Než odpovíš, nejprve vysvětli svůj postup uvažování krok za krokem
v <thinking> tagu. Teprve poté formuluj finální odpověď v <answer> tagu.
```

7.1.3.2 5. Best-of-N verifikace

Spusťte Claude přes stejný prompt vícekrát a porovnejte výstupy. Nekonzistence mezi výstupy indikují halucinace. Tato technika je relevantní hlavně pro API/automatizované pipeline.

7.1.3.3 6. Iterativní zpřesnění

Použijte výstup Claude jako vstup pro follow-up prompt, kde ho požádáte o verifikaci nebo rozšíření předchozích tvrzení.

```
[první volání → Claude vygeneruje odpověď]

[druhé volání:]
Toto je tvoje předchozí odpověď: {{odpověď}}.
Zkontroluj každé tvrzení. Pokud některé nemůžeš ověřit
na základě poskytnutého dokumentu, označ ho jako neověřené.
```

7.1.3.4 7. Omezení externích znalostí

Explicitně instruujte Claude, aby používal POUZE informace z poskytnutých dokumentů.

```
Odpovídej VÝHRADNĚ na základě informací v přiloženém dokumentu.
NEPOUŽÍVEJ své obecné znalosti. Pokud dokument odpověď neobsahuje,
řekni to.
```

7.1.4 Specificky pro Claude Code (agentic coding)

Modely Claude 4.x jsou výrazně méně náchylné k halucinacím v kódu. Pro další posílení:

```
<investigate_before_answering>
Nikdy nespekuluj o kódu, který jsi neotevřel.
Pokud uživatel odkazuje na konkrétní soubor, MUSÍŠ ho přečíst
před odpovědí. Ujisti se, že prozkoumáš relevantní soubory
PŘEDTÍM, než odpovíš na otázky o codebase.
</investigate_before_answering>
```

V CLAUDE.md tohle funguje jako persistentní instrukce: Claude pak vždy čte soubory předtím, než o nich mluví.

7.2 Část 2: Zvýšení konzistence výstupu

Konzistence = Claude produkuje výstup ve stejném formátu, stylu a kvalitě při opakovaném volání. Klíčové pro automatizované pipeline, chatboty a produkční aplikace.

7.2.1 Techniky pro konzistentní výstup

7.2.1.1 1. Přesná definice výstupního formátu

Přesně specifikujte požadovaný formát — JSON, XML, vlastní šablony. Claude musí rozumět každému elementu.

Špatně:

```
Analyzuj tento text a vrať výsledky.
```

Dobře:

```
Analyzuj tento text a vrať výsledek jako JSON v tomto přesném formátu:
{
  "sentiment": "positive" | "negative" | "neutral",
  "confidence": 0.0-1.0,
  "key_topics": ["topic1", "topic2"],
  "summary": "max 2 věty"
```

```
}
Nepiš nic jiného než tento JSON.
```

7.2.1.2 2. Structured Outputs (API)

Pokud potřebujete **garantovaně** validní JSON podle schématu, použijte Structured Outputs místo prompt engineeringu. Structured outputs poskytují garantovanou shodu se schématem a jsou přímo navrženy pro tento účel.

Pro tool use přidejte `strict: true` do definice nástrojů — Claude pak vždy dodržuje schéma, bez type mismatchů nebo chybějících polí.

7.2.1.3 3. Few-shot příklady (multishot prompting)

Příklady jsou nejspolehlivější způsob, jak řídit formát, tón a strukturu výstupu. 3–5 dobře zvolených příkladů dramaticky zlepší konzistenci.

Pravidla pro příklady:

- **Relevantní** — zrcadlí skutečný use case
- **Diverzní** — pokrývají edge cases, aby Claude nezachytil nechtěné vzory
- **Strukturované** — zabalené v `<example>` tazích pro oddělení od instrukcí

```
<examples>
<example>
<input>Zákazník: Kde je moje objednávka #12345?</input>
<output>{"intent": "order_status", "order_id": "12345", "sentiment": "neutral"}</
output>
</example>
<example>
<input>Zákazník: Tohle je hrozné! Chci peníze zpět!</input>
<output>{"intent": "refund_request", "order_id": null, "sentiment": "negative"}</
output>
</example>
<example>
<input>Zákazník: Máte slevu na nové produkty?</input>
<output>{"intent": "pricing_inquiry", "order_id": null, "sentiment": "neutral"}</
output>
</example>
</examples>
```

7.2.1.4 4. Retrieval-Augmented Generation (RAG)

Pro úlohy vyžadující konzistentní kontext (chatboti, knowledge bases) použijte retrieval k ukotvení odpovědí v pevně daném informačním setu. Claude pak odpovídá na základě dodaných dat, ne z vlastních znalostí.

7.2.1.5 5. Dekompozice na subtasky

Rozdělte komplexní úlohy na menší, konzistentní podúlohy. Každý subtask dostane Claudeovu plnou pozornost, což snižuje nekonzistentní chyby napříč velkými workflow.

```
Krok 1: Extrahuj všechna jména osob z dokumentu.
Krok 2: Pro každou osobu identifikuj její roli.
Krok 3: Vrať výsledek jako JSON pole.
```

Prompt chaining (řetězení promptů) toto formalizuje: výstup jednoho promptu → vstup dalšího. Každý subtask má jeden jasný cíl.

7.2.1.6 6. System prompt pro udržení role

Pro chatboty a role-based aplikace definujte roli a osobnost v system promptu.

```
system: Jsi zákaznický asistent společnosti ACME.
Vždy odpovídáš zdvořile, profesionálně, v češtině.
Nikdy neodkazuješ na konkurenční produkty.
Pokud neznáš odpověď, naber eskaluj na lidského operátora.
Formát odpovědí: max 3 věty, bez bullet pointů.
```

System prompt nastavuje silný základ pro konzistentní odpovědi i v dlouhých konverzacích.

7.2.1.7 7. Prompt templates (API)

Oddělte fixní obsah (instrukce, pravidla) od variabilního obsahu (uživatelský vstup). Šablony s `{{proměnnými}}` zajišťují konzistentní strukturu napříč interakcemi.

```
Analyzuj následující zákaznickou recenzi a klasifikuj ji.

<review>
{{customer_review}}
</review>

Vrať výsledek v JSON formátu:
{"category": "...", "sentiment": "...", "action_needed": true/false}
```

7.2.2 Specificky pro Claude Code

7.2.2.1 CLAUDE.md jako kotva konzistence

V Claude Code je `CLAUDE.md` nejsilnější nástroj pro konzistenci. Persistentní pravidla se aplikují na každou session:

```
## Konvence kódu
- Vždy TypeScript, strict mode
- Funkce max 30 řádků
- Každá funkce má JSDoc komentář
- Error handling: vždy try-catch s custom error classes
- Testy: pro každou novou funkci unit test v __tests__/
```

7.2.2.2 Skills pro opakovatelné workflow

Skills zajišťují, že Claude provede stejný postup pokaždé stejně — na rozdíl od ad-hoc instrukcí v chatu.

7.3 Shrnutí: Co použít kdy

Situace	Technika
Claude vymýšlí fakta	Povolte „nevím“ + omezení na dodaná data
Nesprávná extrakce z dokumentu	Přímé citáty → odpověď na základě citátů
Nekonzistentní JSON formát	Structured Outputs (API) nebo přesná šablona + příklady
Chatbot mění styl odpovědí	System prompt s detailní rolí + příklady
Komplexní úloha s chybami	Dekompozice na subtasky + prompt chaining
Code review/analýza halucinuje	`<investigate_before_answering>` + čtení souborů před odpovědí
Pipeline potřebuje stejný výstup	Prompt template + few-shot příklady + temperature 0
High-stakes rozhodnutí	Best-of-N verifikace + iterativní zpřesnění

7.3.1 Důležité omezení

Žádná z těchto technik halucinace **zcela neeliminuje**. Pro kritická rozhodnutí vždy validujte klíčové informace nezávisle. Techniky se kombinují — například přímé citáty + omezení externích znalostí + chain-of-thought verifikace dohromady poskytují silnější ochranu než každá zvlášť.

8 Skills v Claude Code — co to je, jak je vytvářet a sdílet

8.1 Co jsou skills

Skills jsou **znovupoužitelné instrukce**, které rozšiřují schopnosti Claude Code. Každý skill je adresář s `SKILL.md` souborem, který Claude čte a aplikuje na práci. Skills fungují jako `/slash-command` v terminálu.

Skills jsou součástí otevřeného standardu **Agent Skills**, který funguje napříč více AI nástroji (nejen Claude Code).

8.2 Struktura skill souboru

Každý `SKILL.md` má dvě části:

```
---
name: explain-code
description: Explains code with visual diagrams and analogies.
  Use when explaining how code works or when the user asks
  "how does this work?"
---

When explaining code, always include:
1. **Start with an analogy**: Compare the code to something from everyday life
2. **Draw a diagram**: Use ASCII art to show the flow
3. **Walk through the code**: Explain step-by-step what happens
4. **Highlight a gotcha**: What's a common mistake?
```

- **YAML frontmatter** (mezi `---`) — metadata: název, popis, řízení invokace
- **Markdown obsah** — instrukce, které Claude následuje

8.2.1 Frontmatter pole

Pole	Popis
`name`	Název ⟶ stane se `/slash-command`
`description`	Popis — Claude ho používá k rozhodnutí, kdy skill automaticky načíst
`disable-model-invocation`	`true` = jen uživatel může spustit (ne Claude automaticky)
`user-invocable`	`false` = jen Claude může použít (ne uživatel přes `/`)
`allowed-tools`	Omezení nástrojů: `Read, Grep, Glob` — Claude smí použít jen tyto
`context`	`fork` = spustit v izolovaném kontextu (subagent)

8.3 Kde skills žijí

8.3.1 Osobní skills (všechny projekty)

```
~/.claude/skills/
├── explain-code/
│   └── SKILL.md
└── commit-message/
    └── SKILL.md
```

8.3.2 Projektové skills (sdílené přes git)

```
<projekt>/.claude/skills/
├── api-conventions/
│   └── SKILL.md
└── deploy/
    ├── SKILL.md
    └── scripts/
        └── deploy.sh
```

Commitněte do version control → kdokoliv klonuje repo, dostane skills automaticky.

8.3.3 Plugin skills (sdílené napříč repozitáři)

```
my-plugin/
├── .claude-plugin/
│   └── plugin.json
└── skills/
    ├── code-review/
    │   └── SKILL.md
    └── pdf-processor/
        ├── SKILL.md
        └── scripts/
```

8.3.4 Enterprise skills (organizace)

Administrátoři mohou deploynout skills pro celou organizaci přes managed settings.

8.4 Dva typy obsahu skills

8.4.1 1. Referenční obsah (znalosti)

Claude aplikuje znalosti na aktuální práci. Spouští se inline vedle konverzačního kontextu.

```
---
name: api-conventions
description: API design patterns for this codebase
---
When writing API endpoints:
- Use RESTful naming conventions
- Return consistent error formats
- Include request validation
```

8.4.2 2. Task obsah (workflow)

Krok-za-krokem instrukce pro konkrétní akci. Typicky spouštěné manuálně.

```
---
name: fix-issue
description: Fix a GitHub issue
disable-model-invocation: true
---
Analyze and fix the GitHub issue: $ARGUMENTS.
1. Use `gh issue view` to get the issue details
2. Understand the problem described in the issue
3. Search the codebase for relevant files
4. Implement the necessary changes
5. Write and run tests to verify the fix
6. Create a descriptive commit message
7. Push and create a PR
```

Spuštění: `/fix-issue 1234`

8.5 Řízení invokace

Konfigurace	Kdo spouští	Kdy použít
výchozí (nic)	Uživatel i Claude	Většina skills
`disable-model-invocation: true`	Jen uživatel	Workflows se side-effecty: `/deploy`, `/commit`, `/send-slack`
`user-invocable: false`	Jen Claude	Background znalosti, které nejsou akce: legacy-system-context

8.6 Podpůrné soubory

`SKILL.md` může odkazovat na další soubory ve stejném adresáři:

```
deploy/
├── SKILL.md           # Hlavní instrukce (povinné)
├── reference.md       # Detailní dokumentace
├── templates/         # Šablony pro Claude k vyplnění
├── examples/          # Ukázkové výstupy
└── scripts/           # Skripty, které Claude může spouštět
    └── deploy.sh
```

V `SKILL.md` řeknete Claude, aby je četl jen když potřebuje:

```
For detailed API reference, read ./reference.md
For deployment steps, execute ./scripts/deploy.sh
```

Toto je **progressive disclosure** — Claude nenačte vše najednou, ale jen když to potřebuje. Šetří to context window.

8.7 $ARGUMENTS — dynamický vstup

Placeholder `$ARGUMENTS` zachytí text, který uživatel zadá za název skillu:

```
---
name: greet
description: Greet the user with a personalized message
---
Greet the user named "$ARGUMENTS" warmly.
```

Spuštění: `/greet Petr` ⟶ Claude pozdraví Petra.

8.8 Omezení nástrojů (allowed-tools)

Pro bezpečnost nebo izolaci můžete omezit, jaké nástroje Claude smí použít:

```
---
name: safe-reader
description: Read files without making changes
allowed-tools: Read, Grep, Glob
---
```

Když je tento skill aktivní, Claude může použít **pouze** Read, Grep a Glob — bez editace, bez bashe.

8.9 Best practices

8.9.1 SKILL.md by měl být pod 500 řádků

Detailní reference přesuňte do podpůrných souborů.

8.9.2 Popis musí být specifický

Špatně: `Helps with PDFs`
Dobře: `Extract data from PDF forms, fill PDF templates, and merge PDF documents. Use when working with PDF files, document extraction, or form processing.`

Popis pojmenuje konkrétní akce a obsahuje klíčová slova, která uživatel přirozeně použije.

8.9.3 Progressive disclosure šetří kontext

Hlavní instrukce v SKILL.md, detaily v podpůrných souborech. Claude je načte on-demand.

8.9.4 Testujte s debug módem

```
claude --debug
```

Uvidíte chyby při načítání skills.

8.9.5 Live editing

Skills v `.claude/skills/` přidaných přes `--add-dir` se automaticky detekují při změně — nemusíte restartovat Claude Code.

8.10 Skills vs. CLAUDE.md vs. MCP

	Skills	CLAUDE.md	MCP servery
Co dělá	Specifické instrukce a workflows	Persistentní pravidla a kontext	Připojení k externím službám
Kdy se načte	On-demand (automaticky nebo `/`)	Vždy na začátku session	Vždy (tool definitions)
Context cost	Nízký (jen popisy, obsah on-demand)	Celý obsah vždy	Tool schemas vždy
Vhodné pro	Opakovatelné postupy, znalosti	Konvence, pravidla, stav projektu	GitHub, DB, Slack, API
Sdílení	Git, pluginy, enterprise	Git	Konfigurace (settings.json)

9 Sub-agenti v Claude Code — co to jsou, jak je vytvářet a používat

9.1 Co jsou sub-agenti

Sub-agenti jsou **specializovaní AI asistenti**, které Claude Code může spouštět pro konkrétní typy úkolů. Každý sub-agent má vlastní system prompt, vlastní sadu nástrojů a — klíčově — **vlastní context window oddělený od hlavní konverzace**.

Představte si to jako tým specialistů: Claude (hlavní vlákno) je projektový manažer, který ví, kdy zavolat code reviewera, debuggera nebo datového analytika. Každý specialista pracuje izolovaně a vrátí výsledek zpět.

9.2 Jak sub-agenti fungují

Když Claude narazí na úkol odpovídající popisu některého sub-agenta, může ho delegovat. Sub-agent úkol zpracuje ve svém vlastním kontextu a vrátí výsledek. Hlavní konverzace tím nezaplní svůj context window detaily podúkolu.

Spuštění probíhá dvěma způsoby:

Automaticky — Claude sám rozhodne na základě popisu (`description`) agenta a aktuálního kontextu.

Explicitně — vy řeknete Claudovi, kterého agenta použít:

```
> Použij code-reviewer sub-agenta na moje poslední změny
> Nech debugger sub-agenta prozkoumat tuto chybu
```

9.3 Struktura souboru sub-agenta

Sub-agenti jsou Markdown soubory s YAML frontmatter — stejný princip jako u skills. Rozdíl je v umístění a v tom, co frontmatter obsahuje.

```
---
name: code-reviewer
description: Expert code reviewer. Proactively reviews code for quality,
  security, and maintainability. Use PROACTIVELY after writing or modifying code.
tools: Read, Grep, Glob, Bash
model: sonnet
---

Jsi senior code reviewer zaměřený na kvalitu a bezpečnost kódu.

Při spuštění:
1. Spusť `git diff` a zjisti poslední změny
2. Zkontroluj upravené soubory
3. Zhodnoť čitelnost, error handling, bezpečnost a pokrytí testy

Výsledky seřaď podle závažnosti: kritické → varování → návrhy.
```

9.3.1 Pole frontmatter

Pole	Popis
`name`	Jedinečný identifikátor (malá písmena, pomlčky)
`description`	Popis účelu — Claude ho používá při automatické delegaci

`tools`	Seznam nástrojů. Pokud chybí, agent zdědí vše z hlavního vlákna
`model`	`sonnet`, `opus`, `haiku` nebo `inherit`. Výchozí: `sonnet`
`permissionMode`	Jak agent řeší povolení: `default`, `acceptEdits`, `bypassPermissions`, `plan`
`skills`	Skills, které se automaticky načtou při spuštění agenta

9.4 Kde sub-agenti žijí

9.4.1 Projektový agent (sdílený přes git)

```
<projekt>/.claude/agents/
└── code-reviewer.md
```

Committujte do version control — celý tým ho má automaticky k dispozici.

9.4.2 Osobní agent (všechny projekty)

```
~/.claude/agents/
└── code-reviewer.md
```

Pokud existuje agent stejného jména na obou místech, projektový má přednost.

9.5 Jak vytvořit sub-agenta

9.5.1 Doporučená cesta: přes `/agents`

```
/agents
```

Otevře se interaktivní menu, kde zvolíte:

1. **Umístění** — projektový (`.claude/agents/`) nebo osobní (`~/.claude/agents/`)
2. **Způsob vytvoření** — nechat vygenerovat Claudem (doporučeno) nebo vyplnit ručně
3. **Nástroje** — menu zobrazí všechny dostupné nástroje včetně MCP

Výsledný soubor si pak upravte v editoru podle potřeby.

9.5.2 Manuálně — přímo souborem

```
mkdir -p .claude/agents

cat > .claude/agents/test-runner.md << 'EOF'
---
name: test-runner
description: Use proactively to run tests and fix failures after code changes.
tools: Bash, Read, Grep
---

Jsi expert na testování. Po každé změně kódu spusť příslušné testy.
Pokud testy selžou, analyzuj chyby a oprav je — bez změny záměru testů.
EOF
```

Sub-agenti vytvořeni ručně se načtou až při příštím startu session. Pro okamžité použití využijte `/agents`.

9.6 Výběr modelu

Pole `model` vám dává přímou kontrolu nad náklady a výkonem:

```
model: haiku    # Rychlý, levný – vhodný pro jednoduché úkoly (průzkum kódu, grep)
model: sonnet   # Výchozí – dobrý poměr výkon/cena pro většinu agentů
model: opus     # Nejvýkonnější – rezervujte pro náročné analytické úkoly
model: inherit  # Použije stejný model jako hlavní konverzace
```

Pokud pole vynecháte, Claude Code použije výchozí model pro sub-agenty — typicky `sonnet`.

9.7 Vestavění sub-agenti

Claude Code má tři zabudované agenty, které pracují automaticky na pozadí:

general-purpose — pro složité vícekrokové úkoly vyžadující čtení i zápis souborů. Používá Sonnet.

explore — pro rychlé prohledávání kódu v read-only režimu. Používá Haiku, je výrazně rychlejší a levnější.

plan — pro průzkum kódu v plan módu, kdy Claude sbírá kontext před tím, než navrhne postup. Nemůže měnit soubory.

Tyto agenty nemusíte konfigurovat — Claude je volí sám.

9.8 Řízení nástrojů

Stejně jako u skills, zde platí princip nejmenšího oprávnění:

```
# Agent má přístup jen ke čtení – nemůže nic změnit
tools: Read, Grep, Glob

# Agent pro opravy testů – potřebuje i zápis a bash
tools: Read, Edit, Bash, Grep, Glob
```

Pokud pole `tools` vynecháte, agent zdědí všechny nástroje z hlavního vlákna včetně MCP serverů. To je pohodlné, ale méně bezpečné.

9.9 Klíčový rozdíl od skills

	Skills	Sub-agenti
Umístění	`.claude/skills/`	`.claude/agents/`
Co přidávají	Instrukce a znalosti Claudovi	Samostatného agenta s vlastním kontextem
Context window	Sdílený s hlavní konverzací	Vlastní — izolovaný
Spuštění	`/skill-name` nebo automaticky	Automaticky nebo explicitní žádostí
Vhodné pro	Opakovatelné postupy, konvence	Specializované úkoly, komplexní analýzy

9.10 Best practices

9.10.1 Jeden agent = jedna odpovědnost

Vytvořte `code-reviewer.md` pro review a `debugger.md` pro debugging — ne jednoho všeuměla. Specializace zlepšuje výsledky.

9.10.2 Description musí být konkrétní a akční

Špatně: `Helps with code`

Dobře: `Expert code reviewer. Proactively reviews code for quality, security, and maintainability. Use PROACTIVELY after writing or modifying code.`

Klíčová slova jako `PROACTIVELY` nebo `MUST BE USED` signalizují Claudovi, aby agenta volal aktivně — bez čekání na váš pokyn.

9.10.3 Nechte Claudovi generovat základ

Popište agenta v přirozené češtině nebo angličtině a nechte Claude vygenerovat system prompt. Pak ho dolaďte. Je to rychlejší než psát vše od nuly.

9.10.4 Verzujte projektové agenty

Soubory v `.claude/agents/` patří do gitu — celý tým pak pracuje se stejnými agenty a může je společně vylepšovat.

10 Artifacts v Claude — co to je, jak je vytvářet a sdílet

> **UPOZORNĚNÍ**
>
> Tento materiál se týká funkce Artifacts v **claude.ai** (webové rozhraní), nikoliv Claude Code (CLI). Artifacts nejsou součástí Claude Code.

10.1 Co jsou artifacts

Artifacts jsou **samostatné, interaktivní kusy obsahu**, které Claude vytváří během konverzace. Zobrazují se v dedikovaném panelu vedle chatu a umožňují živý náhled, iteraci a sdílení. Jsou dostupné v claude.ai (web), Claude Desktop a mobilních aplikacích.

Claude vytvoří artifact automaticky, když obsah splňuje dvě podmínky:

- Je **významný a samostatný** (typicky > 15 řádků)
- Je to něco, co budete pravděpodobně chtít **editovat, iterovat nebo použít mimo konverzaci**

10.2 Podporované typy artifacts

Typ	MIME type	Renderuje se?	Popis
React komponenty	`application/vnd.ant.react`	interaktivně	Plně funkční React s hooks, state management, Tailwind CSS
HTML stránky	`text/html`	jako webová stránka	HTML + CSS + JS v jednom souboru, sandbox iframe
SVG obrázky	`image/svg+xml`	vizuálně	Vektorová grafika, diagramy, ilustrace
Mermaid diagramy	`application/vnd.ant.mermaid`	vizuálně	Flowcharty, sequence diagramy, ER diagramy, Gantt
Markdown dokumenty	`text/markdown`	formátovaně	Dokumenty, reporty, články
Code snippety	`application/vnd.ant.code`	syntax highlight	Kód v jakémkoliv jazyce — Python, SQL, JS, Bash…

10.2.1 Dostupné knihovny v React artifacts

React artifacts mohou importovat vybrané knihovny z CDN:

- `lucide-react` — ikony
- `recharts` — grafy a vizualizace
- `mathjs` — matematické operace
- `lodash` — utility funkce
- `d3` — datové vizualizace
- `three.js` (r128) — 3D grafika
- `papaparse` — CSV parsing
- `sheetjs` — Excel soubory
- `chart.js`, `tone`, `mammoth`, `tensorflow`
- `shadcn/ui` — UI komponenty

Externí skripty lze importovat pouze z `https://cdnjs.cloudflare.com`.

10.3 Vytváření artifacts

10.3.1 Doporučený postup

1. **Popište problém**, ne řešení — nechte Claude navrhnout přístup
2. **Nechte Claude klást otázky** — upřesní vaši vizi
3. **Požádejte o vytvoření** — „Can you create this for me?“ / „Let's build this now.”
4. **Iterujte** — „make the buttons bigger“, „add a timer”, „change the color scheme“

10.3.2 Tipy pro efektivní práci

- **Iterujte follow-up prompty** — každý požadavek staví na předchozí verzi
- **Editujte předchozí zprávy** — vytvoříte alternativní větev konverzace s vlastní sadou artifacts
- **Opravujte chyby** — tlačítko „Try fixing with Claude“ u error message automaticky předá detaily chyby
- **Kombinujte typy** — HTML + React, Mermaid pro flow + Markdown pro dokumentaci

10.3.3 Inline editace (od října 2025)

Claude nyní inteligentně volí mezi:

- **Cílená inline editace** — pro malé změny (3–4× rychlejší)
- **Kompletní přegenerování** — jen když je to nutné

10.4 Pokročilé funkce

10.4.1 AI-powered artifacts

Artifacts mohou obsahovat **volání Claude API** — vytvoříte tak interaktivní AI aplikace přímo v artefaktu. Uživatelé vaší sdílené aplikace nepotřebují API klíč — usage se počítá proti jejich vlastnímu Claude předplatnému.

Dostupné na všech plánech (free, Pro, Max, Team, Enterprise).

10.4.2 MCP integrace v artifacts

Artifacts se mohou připojit k externím službám přes MCP:

- Google Calendar, Asana, Slack a další
- Vlastní MCP servery, které máte nakonfigurované
- Uživatel schvaluje přístup při prvním použití

Dostupné na Pro, Max, Team a Enterprise plánech.

10.4.3 Persistent storage

Artifacts mohou ukládat a načítat data přes `window.storage` API:

- **Osobní data** (`shared: false`) — vidí jen aktuální uživatel
- **Sdílená data** (`shared: true`) — vidí všichni uživatelé artefaktu
- Funguje pouze u **publikovaných** artifacts
- Key-value store, max 5 MB per klíč

Dostupné na Pro, Max, Team a Enterprise plánech.

10.5 Sdílení artifacts

10.5.1 Publishing (Free, Pro, Max)

Publikování zpřístupní artifact veřejně — kdokoliv s odkazem ho může vidět a používat.

1. Přejděte na artifact, zkontrolujte správnou verzi
2. Klikněte **„Publish“**
3. Zkopírujte veřejný odkaz

Komu je přístupný:

- **Bez účtu** — může prohlížet a interagovat (základní funkce)
- **S účtem** — plný přístup včetně „Customize“ (vytvoření vlastní kopie)
- **AI-powered funkce** — vyžadují přihlášení, usage jde proti limitu uživatele

10.5.2 Embed na webu

Po publikování je k dispozici tlačítko **„Get embed code“**:

- Vygeneruje `<iframe>` kód pro vložení na web
- Musíte specifikovat **povolené domény**

10.5.3 Sharing (Team, Enterprise)

Interní sdílení — artifact je dostupný pouze členům organizace.

1. Klikněte **„Share“**
2. Klikněte **„Share & copy link“**
3. Odkaz funguje jen pro přihlášené členy organizace

Omezení:

- Artifacts z Team/Enterprise účtů **nelze publikovat veřejně**
- Pokud artifact vznikl v projektu, příjemce musí mít přístup k projektu
- Sdílení zpřístupní i přílohy a soubory z konverzace

10.5.4 Unpublishing

- Po unpublishnutí **nelze znovu publikovat** tentýž artifact
- Smaže se veškeré persistent storage data
- Pro opětovné sdílení musíte vytvořit nový artifact

10.6 Customizing (remix)

Dostupné na všech plánech. Umožňuje vytvořit vlastní kopii cizího artefaktu:

1. Na zobrazeném artefaktu klikněte **„Customize“**
2. Otevře se nová konverzace s obsahem artefaktu
3. Můžete ho upravovat a iterovat — originál zůstává nezměněný

10.7 Artifacts space

V levém sidebaru Claude je dedikovaný prostor **Artifacts**:

- Přehled všech vašich artifacts
- **Inspiration tab** (Free, Pro, Max) — kurátorská kolekce artifacts v kategoriích (Learn something, Life hacks, Play a game, Be creative, Touch grass)
- Team/Enterprise uživatelé vidí artifacts sdílené kolegy místo veřejné Inspiration tab

10.8 Artifacts vs. Claude Code file creation

	Artifacts (claude.ai)	**File creation (Claude Code)**
Prostředí	Webové rozhraní, iframe sandbox	Terminál, reálný souborový systém
Živý náhled	Okamžitý v panelu	Nutné otevřít soubor externě
Interaktivita	React, HTML, AI-powered	Plný přístup k OS, bash, Python…
Sdílení	Publish/Share jedním klikem	Git, manuální sdílení
Persistence	Persistent storage API	Soubory na disku
Vhodné pro	Prototypy, vizualizace, demíčka	Produkční kód, komplexní projekty